

HW4 Report
梁正 R13922213

1. Implementation

Host (CPU) Responsibilities:

- Data Preparation: The solve function, running on the host, is responsible for all preliminary setup. It reads block information (version, previous hash, nbits, ntime, and merkle branches) from an input file.
- Merkle Root Calculation: The host code calculates the Merkle root from the provided transaction branches. This calculation is performed sequentially on the CPU as it is a one-time setup task for each block and not the primary computational bottleneck.
- Block and Target Setup: It assembles a HashBlock structure with all the header information except for the nonce. It also calculates the 256-bit target value from the compact nbits representation.
- GPU Data Transfer: Before launching the kernel, the host transfers the necessary data to the GPU.
 - The HashBlock template and the calculated target are copied to `__constant__` memory on the device using `cudaMemcpyToSymbol`. Constant memory is used because this data is read-only and accessed uniformly by all threads, making it a fast, cached memory space ideal for this purpose.
 - Global memory is allocated on the device for the results: `d_found_nonce` to store the successful nonce and `d_found_hash` for the corresponding block hash. `d_found_nonce` is initialized to `0xffffffffU` as a sentinel value, indicating that no solution has been found yet.
- Kernel Launch: The host launches the `mining_kernel` on the GPU with a grid of thread blocks (`BLOCKS_PER_GRID x THREADS_PER_BLOCK`). This initiates the parallel search for the nonce.
- Result Retrieval: After the kernel completes (enforced by `cudaDeviceSynchronize()`), the host copies the `d_found_nonce` and `d_found_hash` from the GPU back to host memory.
- Output and Cleanup: Finally, the host prints the results, writes the found nonce to the output file, and releases the allocated GPU memory.

Device (GPU) Responsibilities:

- Parallel Hashing (`mining_kernel`): This is the `__global__` function executed by thousands of GPU threads in parallel.

- Nonce-Space Partitioning: The entire 4-billion (2^{32}) nonce space is divided among all active threads. Each thread calculates its unique global ID and determines the specific range of nonces it is responsible for checking.
- Iterative Hashing: Each thread iterates through its assigned nonce range. In each iteration, it:
 - i. Updates its local copy of the HashBlock with the current nonce.
 - ii. Performs a double SHA-256 hash on the 80-byte block header using the `double_sha256_device` function.
 - iii. Compares the resulting hash with the `d_target` value.
- Device-Side Functions: The SHA-256 algorithm is implemented entirely on the device using `__device__` functions (`sha256_transform_device`, `sha256_device`, `double_sha256_device`). This avoids the high latency of calling back to the host for hashing operations.
- Solution Handling:
 - If a thread finds a hash that is less than the target, it has found a valid solution.
 - It uses atomicCAS (atomic Compare-And-Swap) to write its successful nonce to the global `d_found_nonce` variable. This atomic operation ensures that only the very first thread to find a solution can update the variable from its initial sentinel value, preventing race conditions.
 - The thread then writes the successful hash to `d_found_hash` and terminates. Other threads, upon checking `d_found_nonce` at the start of their next loop, will see it has been changed and will also terminate early.

2. Parallelization and Optimization Techniques

- Offloading Computation to GPU: The most computationally expensive part of the mining process is the repeated application of the double SHA-256 hash function. The parallel code moves this entire workload to the GPU by implementing the SHA-256 algorithm in `__device__` functions. This leverages the GPU's specialized hardware for high-throughput arithmetic operations.
- Use of Constant Memory (`__constant__`): The block header template (`d_block`), the target hash (`d_target`), and the SHA-256 constants (`d_k`) are identical for every hash attempt within a single block's search. Storing this data in `__constant__` memory allows for efficient broadcasting to all threads. When threads in a warp access the same address in constant

memory, the data is broadcast to all of them in a single transaction, significantly reducing memory latency compared to global memory access.

- **Early Exit Mechanism:** The brute-force search can be stopped as soon as any thread finds a valid nonce.
- **Atomic Operations:** The use of atomicCAS provides a thread-safe way for the first successful thread to "claim" the solution by updating a global flag (d_found_nonce).
- **Work Reduction:** In the main loop of the kernel, each thread checks the value of d_found_nonce before starting a new hash calculation. If the value is no longer the initial sentinel (0xffffffffU), the thread knows a solution has been found elsewhere and immediately terminates. This prevents the GPU from performing a significant amount of redundant work after a solution is already discovered.
- **In-Kernel Logic:** The entire hash-and-compare loop is contained within the GPU kernel. This minimizes the communication between the host and device. The host only needs to launch the kernel once and wait for the result, avoiding the massive overhead that would be incurred if each hash result were sent back to the CPU for verification.

3. Experiment

threads per block	blocks per grid	total threads ▼	time (ms)
512	2048	1048576	3341
256	2048	524288	3436
512	1024	524288	3514
128	2048	262144	3680
256	1024	262144	4050
512	512	262144	3872
128	1024	131072	4386
256	512	131072	4542
128	512	65536	4693

Total Threads vs Execution Time (case00)

